

Assignment-11.1

Data Structures with AI: Implementing Fundamental Structures

Sarika Palle

2403a51l33

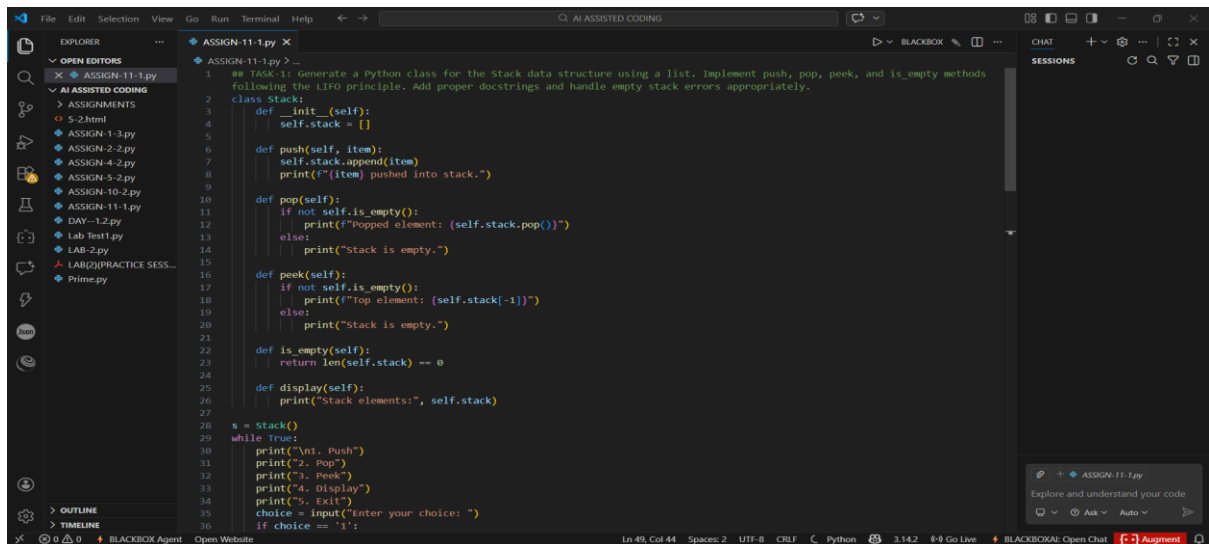
B-52

Task Description #1 – Stack Implementation:

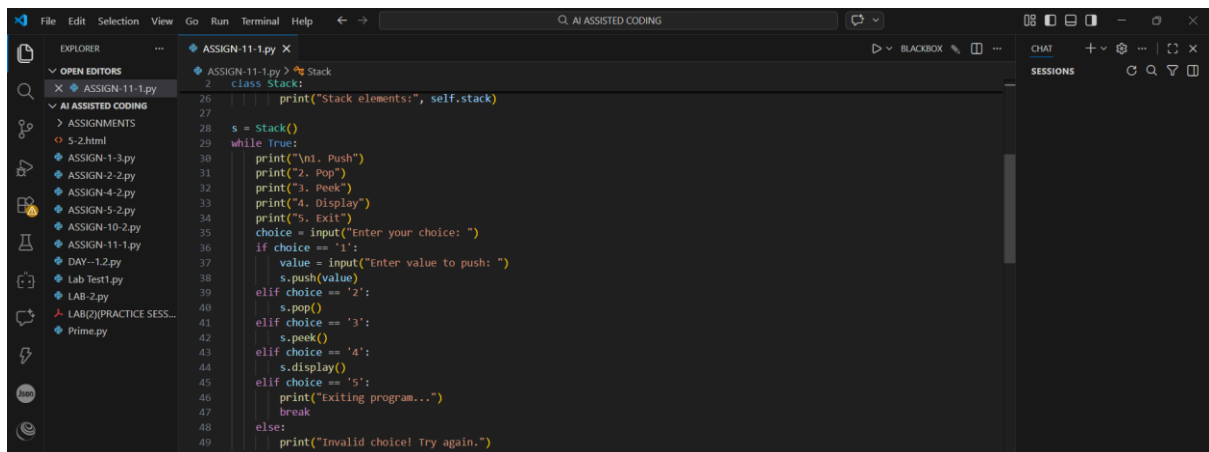
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

PROMPT: Generate a Python class for the Stack data structure using a list. Implement push, pop, peek, and is_empty methods following the LIFO principle. Add proper docstrings and handle empty stack errors appropriately.

Sample Input Code:

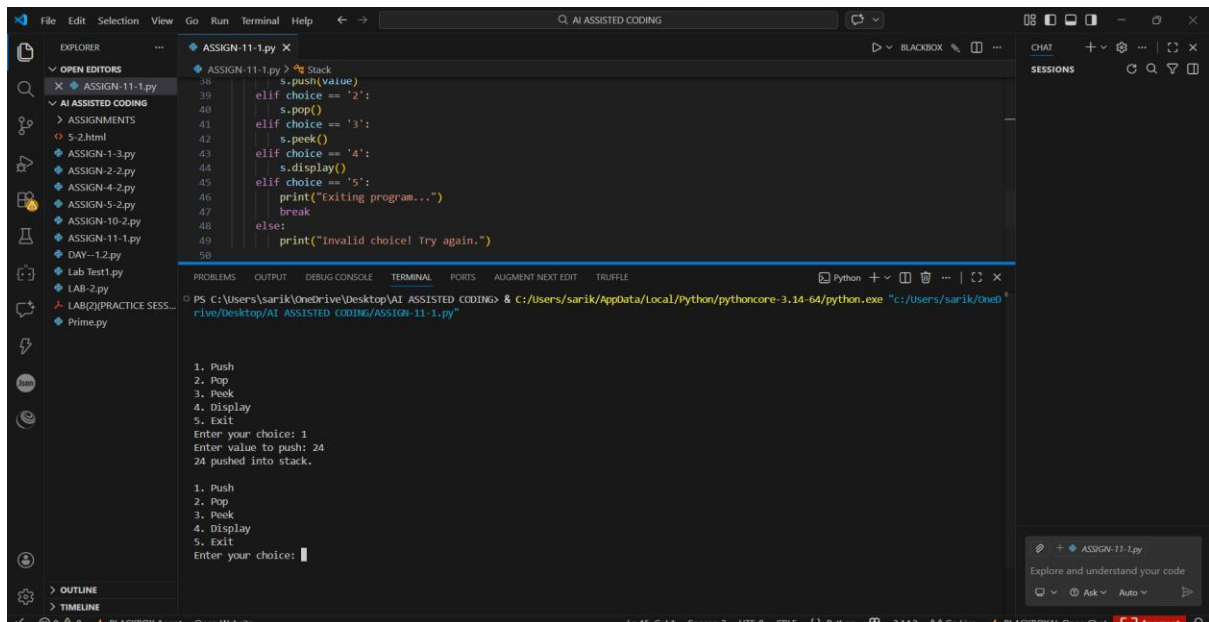


```
1  ## TASK-1: Generate a Python class for the Stack data structure using a list. Implement push, pop, peek, and is_empty methods
2  following the LIFO principle. Add proper docstrings and handle empty stack errors appropriately.
3  class Stack:
4      def __init__(self):
5          self.stack = []
6
7      def push(self, item):
8          self.stack.append(item)
9          print(f"{item} pushed into stack.")
10
11     def pop(self):
12         if not self.is_empty():
13             print(f"Popped element: {self.stack.pop()}")
14         else:
15             print("Stack is empty.")
16
17     def peek(self):
18         if not self.is_empty():
19             print(f"Top element: {self.stack[-1]}")
20         else:
21             print("Stack is empty.")
22
23     def is_empty(self):
24         return len(self.stack) == 0
25
26     def display(self):
27         print("Stack elements:", self.stack)
28
29     s = Stack()
30     while True:
31         print("\n1. Push")
32         print("2. Pop")
33         print("3. Peek")
34         print("4. Display")
35         print("5. Exit")
36         choice = input("Enter your choice: ")
37         if choice == '1':
38             value = input("Enter value to push: ")
39             s.push(value)
40         elif choice == '2':
41             s.pop()
42         elif choice == '3':
43             s.peek()
44         elif choice == '4':
45             s.display()
46         elif choice == '5':
47             print("Exiting program...")
48             break
49         else:
50             print("Invalid choice! Try again.")
```



```
1  class Stack:
2      def __init__(self):
3          self.stack = []
4
5      def push(self, item):
6          self.stack.append(item)
7          print(f"{item} pushed into stack.")
8
9      def pop(self):
10         if not self.is_empty():
11             print(f"Popped element: {self.stack.pop()}")
12         else:
13             print("Stack is empty.")
14
15     def peek(self):
16         if not self.is_empty():
17             print(f"Top element: {self.stack[-1]}")
18         else:
19             print("Stack is empty.")
20
21     def is_empty(self):
22         return len(self.stack) == 0
23
24     def display(self):
25         print("Stack elements:", self.stack)
26
27     s = Stack()
28     while True:
29         print("\n1. Push")
30         print("2. Pop")
31         print("3. Peek")
32         print("4. Display")
33         print("5. Exit")
34         choice = input("Enter your choice: ")
35         if choice == '1':
36             value = input("Enter value to push: ")
37             s.push(value)
38         elif choice == '2':
39             s.pop()
40         elif choice == '3':
41             s.peek()
42         elif choice == '4':
43             s.display()
44         elif choice == '5':
45             print("Exiting program...")
46             break
47         else:
48             print("Invalid choice! Try again.")
```

OUTPUT:



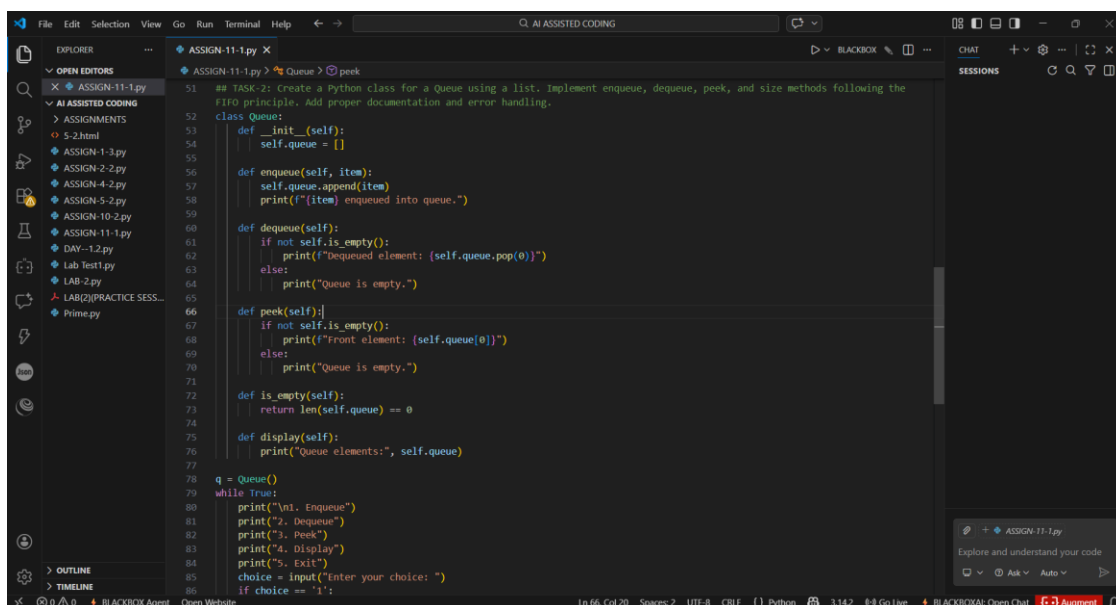
EXPLANATION: A Stack is a linear data structure that follows the LIFO (Last In First Out) principle, where the last element inserted is the first one removed. Operations such as push, pop, and peek are performed at one end called the top. It is commonly used in function calls, undo operations, and expression evaluation.

Task Description #2 – Queue Implementation:

Task: Use AI to implement a Queue using Python lists.

PROMPT: Create a Python class for a Queue using a list. Implement enqueue, dequeue, peek, and size methods following the FIFO principle. Add proper documentation and error handling.

Sample Input Code:



```
77
78 q = Queue()
79 while True:
80     print("\n1. Enqueue")
81     print("2. Dequeue")
82     print("3. Peek")
83     print("4. Display")
84     print("5. Exit")
85     choice = input("Enter your choice: ")
86     if choice == '1':
87         value = input("Enter value to enqueue: ")
88         q.enqueue(value)
89     elif choice == '2':
90         q.dequeue()
91     elif choice == '3':
92         q.peek()
93     elif choice == '4':
94         q.display()
95     elif choice == '5':
96         print("Exiting program...")
97         break
98     else:
99         print("Invalid choice! Try again.")
100
101
```

OUTPUT:

```
83     print("4. Display")
84     print("5. Exit")
85     choice = input("Enter your choice: ")
86     if choice == '1':
87         value = input("Enter value to enqueue: ")
88         q.enqueue(value)
89     elif choice == '2':
90         q.dequeue()
91     elif choice == '3':
92         q.peek()
93     elif choice == '4':
94         q.display()
95     elif choice == '5':
```

```
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to enqueue: 24
24 enqueued into queue.

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 
```

EXPLANATION: A Queue is a linear data structure that follows the FIFO (First In First Out) principle, where the first inserted element is removed first. Elements are added at the rear and removed from the front. It is widely used in scheduling systems, buffering, and real-world waiting line applications.

Task Description #3 – Linked List:

Task: Use AI to generate a Singly Linked List with insert and display methods.

PROMPT: Generate a Python implementation of a Singly Linked List including a Node class and LinkedList class. Implement insert and display methods with clear documentation.

Sample Input Code:

```
100
101 ## TASK-3: Generate a Python implementation of a Singly Linked List including a Node class and LinkedList class. Implement
102 insert and display methods with clear documentation.
103 class Node:
104     def __init__(self, data):
105         self.data = data
106         self.next = None
107 class LinkedList:
108     def __init__(self):
109         self.head = None
110
111     def insert(self, data):
112         new_node = Node(data)
113         if self.head is None:
114             self.head = new_node
115             print(f"{data} inserted as head of the list.")
116         else:
117             current = self.head
118             while current.next:
119                 current = current.next
120             current.next = new_node
121             print(f"{data} inserted into the list.")
122
123     def display(self):
124         if self.head is None:
125             print("The linked list is empty.")
126         else:
127             current = self.head
128             elements = []
129             while current:
130                 elements.append(current.data)
131                 current = current.next
132             print("Linked List elements:", " -> ".join(map(str, elements)))
133
134 ll = LinkedList()
135 while True:
136     print("\n1. Insert")
137     print("2. Display")
138     print("3. Exit")
139     choice = input("Enter your choice: ")
140     if choice == '1':
141         value = input("Enter value to insert: ")
142         ll.insert(value)
143     elif choice == '2':
144         ll.display()
145     elif choice == '3':
146         print("Exiting program...")
147         break
148     else:
149         print("Invalid choice! Try again.")
```

```
107 class LinkedList:
108     def __init__(self):
109         self.head = None
110
111     def insert(self, data):
112         new_node = Node(data)
113         if self.head is None:
114             self.head = new_node
115             print(f"{data} inserted as head of the list.")
116         else:
117             current = self.head
118             while current.next:
119                 current = current.next
120             current.next = new_node
121             print(f"{data} inserted into the list.")
122
123     def display(self):
124         if self.head is None:
125             print("The linked list is empty.")
126         else:
127             current = self.head
128             elements = []
129             while current:
130                 elements.append(current.data)
131                 current = current.next
132             print("Linked List elements:", " -> ".join(map(str, elements)))
133
134 ll = LinkedList()
135 while True:
136     print("\n1. Insert")
137     print("2. Display")
138     print("3. Exit")
139     choice = input("Enter your choice: ")
140     if choice == '1':
141         value = input("Enter value to insert: ")
142         ll.insert(value)
143     elif choice == '2':
144         ll.display()
145     elif choice == '3':
146         print("Exiting program...")
147         break
148     else:
149         print("Invalid choice! Try again.")
```

OUTPUT:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-11-1.py"
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 11
11 inserted as head of the list.

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 14
14 inserted into the list.

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value to insert: 24
24 inserted into the list.
```

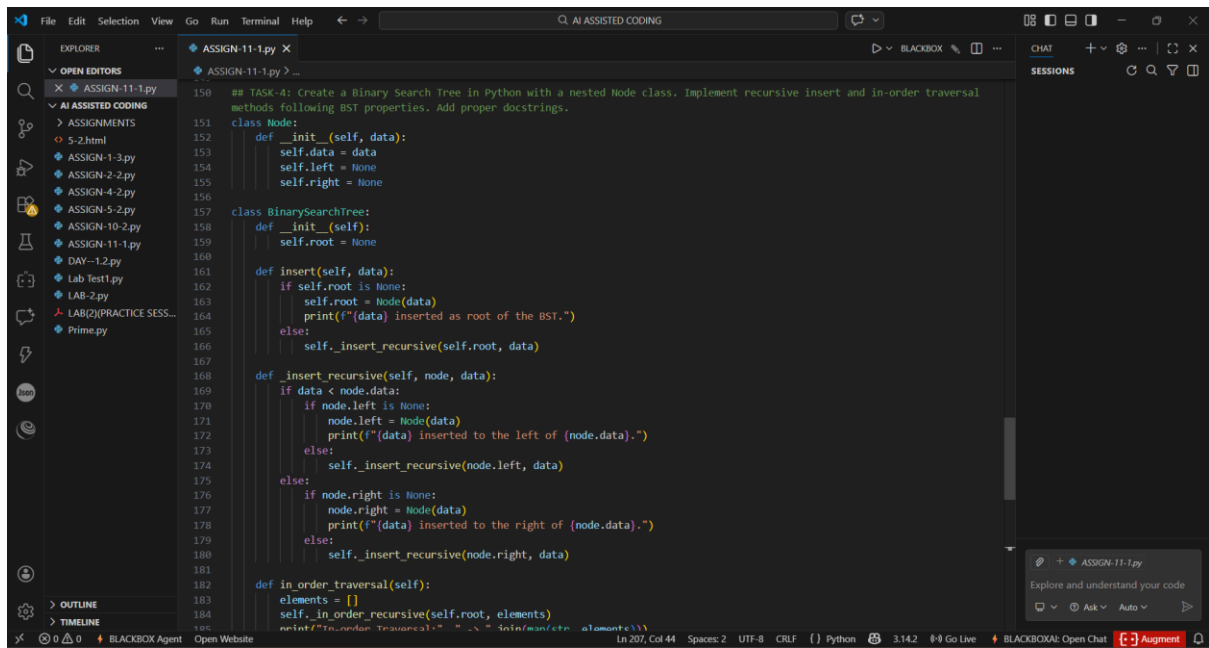
EXPLANATION: A Singly Linked List consists of nodes where each node contains data and a reference to the next node. Unlike arrays, it does not require contiguous memory, making it dynamic in size. It allows efficient insertions and deletions compared to fixed-size structures.

Task Description #4 – Binary Search Tree (BST):

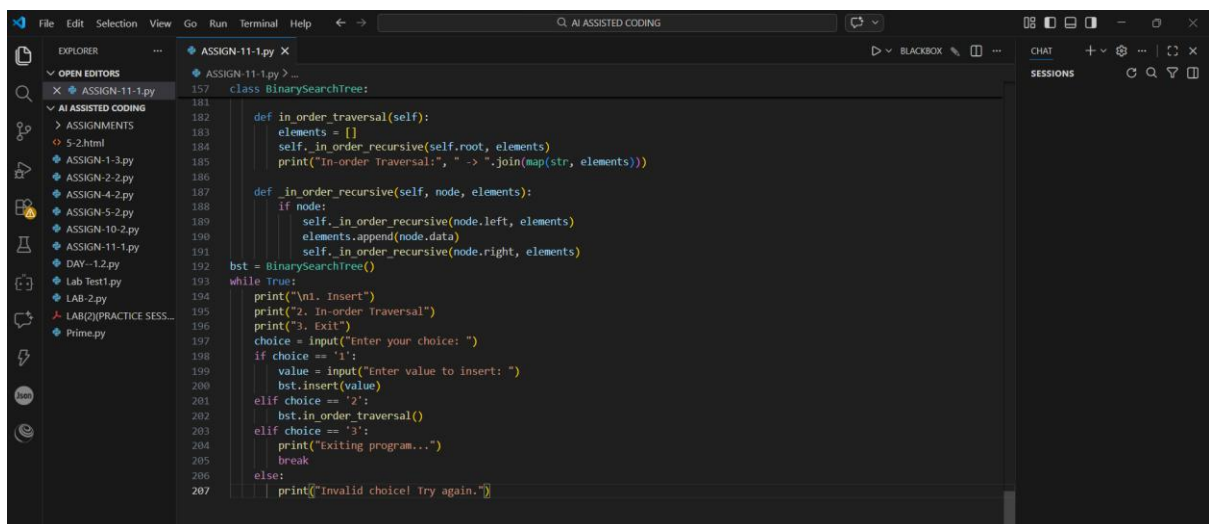
Task: Use AI to create a BST with insert and in-order traversal methods.

PROMPT: Create a Binary Search Tree in Python with a nested Node class. Implement recursive insert and in-order traversal methods following BST properties. Add proper docstrings.

Sample Input Code:

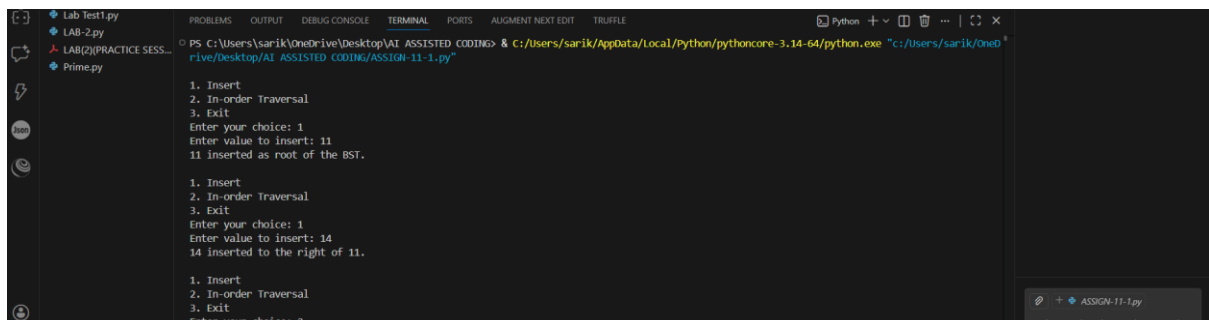


```
150 ## TASK-4: Create a Binary Search Tree in Python with a nested Node class. Implement recursive insert and in-order traversal
151 methods following BST properties. Add proper docstrings.
152
153 class Node:
154     def __init__(self, data):
155         self.data = data
156         self.left = None
157         self.right = None
158
159 class BinarySearchTree:
160     def __init__(self):
161         self.root = None
162
163     def insert(self, data):
164         if self.root is None:
165             self.root = Node(data)
166             print(f"{data} inserted as root of the BST.")
167         else:
168             self._insert_recursive(self.root, data)
169
170     def _insert_recursive(self, node, data):
171         if data < node.data:
172             if node.left is None:
173                 node.left = Node(data)
174                 print(f"{data} inserted to the left of {node.data}.")
175             else:
176                 self._insert_recursive(node.left, data)
177         else:
178             if node.right is None:
179                 node.right = Node(data)
180                 print(f"{data} inserted to the right of {node.data}.")
181             else:
182                 self._insert_recursive(node.right, data)
183
184     def in_order_traversal(self):
185         elements = []
186         self._in_order_recursive(self.root, elements)
187         print(f"In-order Traversal: " + " ".join(map(str, elements)))
```



```
181
182     def in_order_traversal(self):
183         elements = []
184         self._in_order_recursive(self.root, elements)
185         print("In-order Traversal: " + " ".join(map(str, elements)))
186
187     def _in_order_recursive(self, node, elements):
188         if node:
189             self._in_order_recursive(node.left, elements)
190             elements.append(node.data)
191             self._in_order_recursive(node.right, elements)
192
193 bst = BinarySearchTree()
194 while True:
195     print("\n1. Insert")
196     print("2. In-order Traversal")
197     print("3. Exit")
198     choice = input("Enter your choice: ")
199     if choice == '1':
200         value = input("Enter value to insert: ")
201         bst.insert(value)
202     elif choice == '2':
203         bst.in_order_traversal()
204     elif choice == '3':
205         print("Exiting program...")
206         break
207     else:
208         print("Invalid choice! Try again.")
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-11-1.py"

1. Insert
2. In-order Traversal
3. Exit
Enter your choice: 1
Enter value to insert: 11
11 inserted as root of the BST.

1. Insert
2. In-order Traversal
3. Exit
Enter your choice: 1
Enter value to insert: 14
14 inserted to the right of 11.

1. Insert
2. In-order Traversal
3. Exit
Enter your choice: 2
```

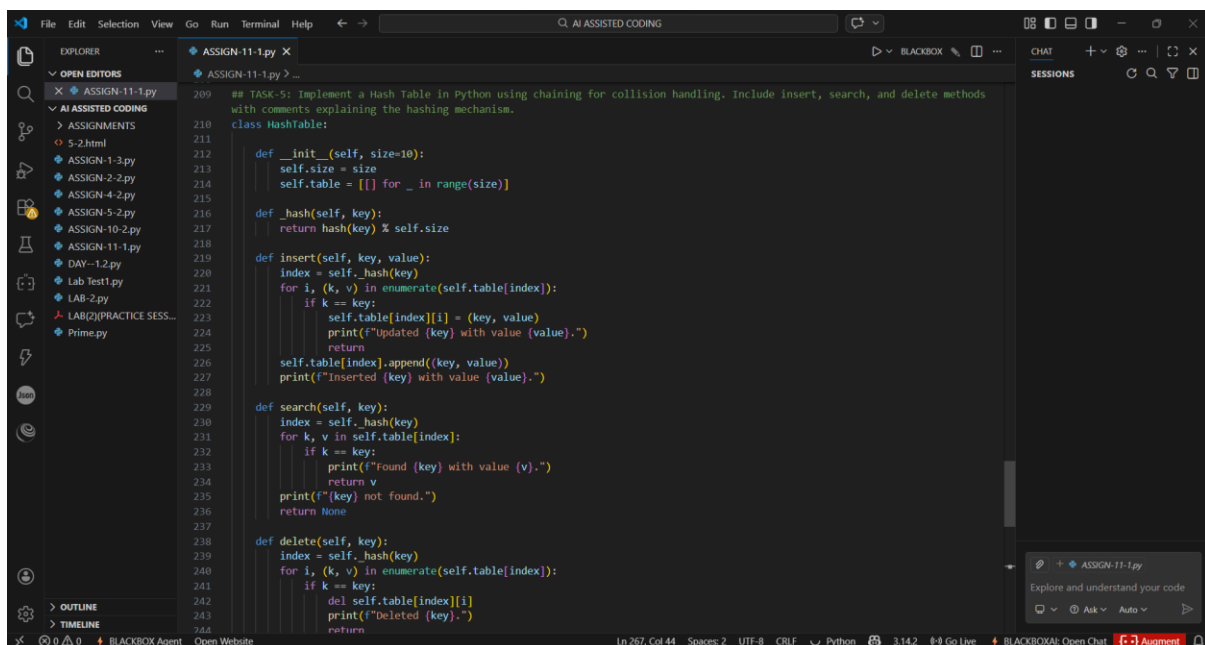
EXPLANATION: A Binary Search Tree is a hierarchical data structure where the left child contains smaller values and the right child contains larger values than the root. This property makes searching, insertion, and deletion efficient. In-order traversal of a BST produces sorted output.

Task Description #5 – Hash Table:

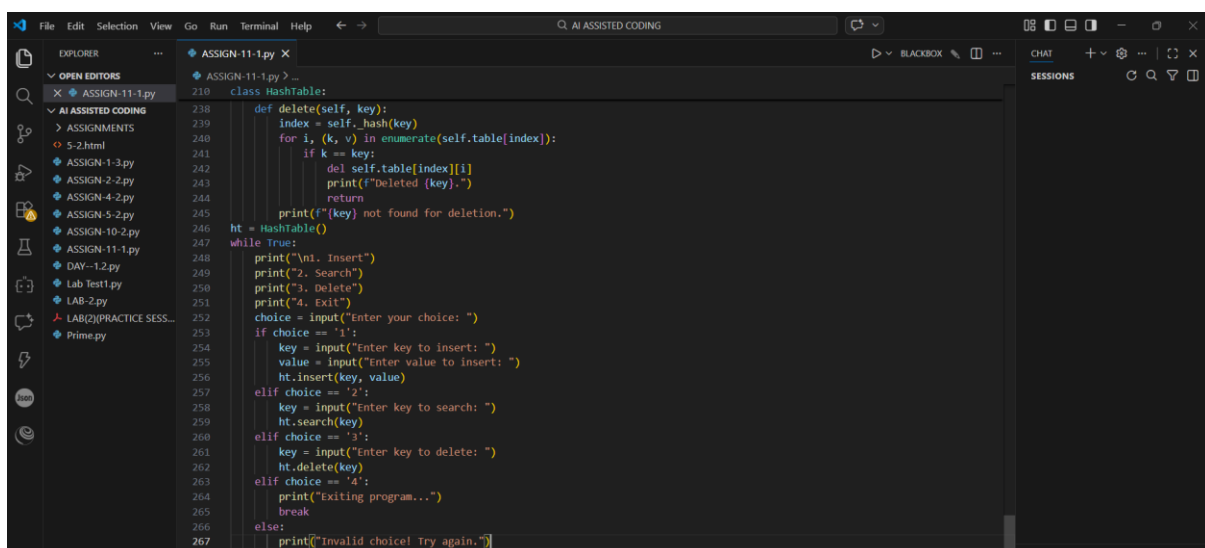
Task: Use AI to implement a hash table with basic insert, search, and delete methods.

PROMPT: Implement a Hash Table in Python using chaining for collision handling. Include insert, search, and delete methods with comments explaining the hashing mechanism.

Sample Input Code:



```
209 ## TASK-5: Implement a Hash Table in Python using chaining for collision handling. Include insert, search, and delete methods
210 with comments explaining the hashing mechanism.
211 class HashTable:
212     def __init__(self, size=10):
213         self.size = size
214         self.table = [[] for _ in range(size)]
215
216     def _hash(self, key):
217         return hash(key) % self.size
218
219     def insert(self, key, value):
220         index = self._hash(key)
221         for i, (k, v) in enumerate(self.table[index]):
222             if k == key:
223                 self.table[index][i] = (key, value)
224                 print(f"Updated {key} with value {value}.")
225                 return
226         self.table[index].append((key, value))
227         print(f"Inserted {key} with value {value}.")
228
229     def search(self, key):
230         index = self._hash(key)
231         for k, v in self.table[index]:
232             if k == key:
233                 print(f"Found {key} with value {v}.")
234                 return v
235         print(f"{key} not found.")
236         return None
237
238     def delete(self, key):
239         index = self._hash(key)
240         for i, (k, v) in enumerate(self.table[index]):
241             if k == key:
242                 del self.table[index][i]
243                 print(f"Deleted {key}.")
244                 return
245         print(f"{key} not found for deletion.")
```



```
246 ht = HashTable()
247 while True:
248     print("\n1. Insert")
249     print("2. Search")
250     print("3. Delete")
251     print("4. Exit")
252     choice = input("Enter your choice: ")
253     if choice == '1':
254         key = input("Enter key to insert: ")
255         value = input("Enter value to insert: ")
256         ht.insert(key, value)
257     elif choice == '2':
258         key = input("Enter key to search: ")
259         ht.search(key)
260     elif choice == '3':
261         key = input("Enter key to delete: ")
262         ht.delete(key)
263     elif choice == '4':
264         print("Exiting program...")
265         break
266     else:
267         print("Invalid choice! Try again.")
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> c:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-11-1.py"

1. Insert
2. Search
3. Delete
4. Exit
Enter your choice: 1
Enter key to Insert: 14
Enter value to insert: 2
Inserted 14 with value 2.

1. Insert
2. Search
3. Delete
4. Exit
Enter your choice: 3
Enter key to delete: 14
Deleted 14.
```

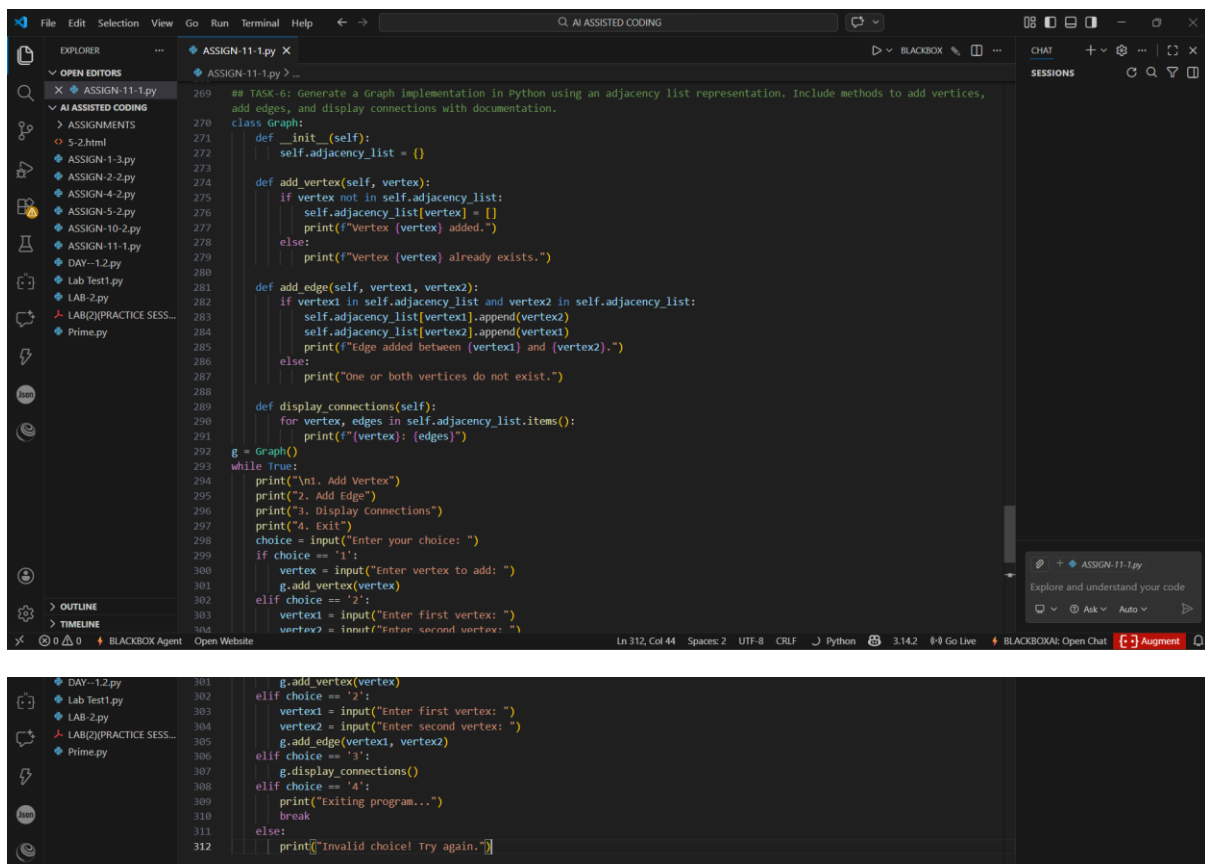
EXPLANATION: A Hash Table stores data in key-value pairs using a hash function to compute an index. It provides fast average-case time complexity for search, insertion, and deletion operations. Collisions are handled using techniques such as chaining.

Task Description #6 – Graph Representation:

Task: Use AI to implement a graph using an adjacency list.

PROMPT: Generate a Graph implementation in Python using an adjacency list representation. Include methods to add vertices, add edges, and display connections with documentation.

Sample Input Code:



```
## TASK-6: Generate a Graph implementation in Python using an adjacency list representation. Include methods to add vertices,
add edges, and display connections with documentation.
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []
            print(f"Vertex {vertex} added.")
        else:
            print(f"Vertex {vertex} already exists.")

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)
            print(f"Edge added between {vertex1} and {vertex2}.")
        else:
            print("One or both vertices do not exist.")

    def display_connections(self):
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex}: {edges}")

g = Graph()
while True:
    print("\n1. Add Vertex")
    print("2. Add Edge")
    print("3. Display Connections")
    print("4. Exit")
    choice = input("Enter your choice: ")
    if choice == '1':
        vertex = input("Enter vertex to add: ")
        g.add_vertex(vertex)
    elif choice == '2':
        vertex1 = input("Enter first vertex: ")
        vertex2 = input("Enter second vertex: ")
        g.add_edge(vertex1, vertex2)
    elif choice == '3':
        g.display_connections()
    elif choice == '4':
        print("Exiting program...")
        break
    else:
        print("Invalid choice! Try again.")
```

OUTPUT:



```
1. Add Vertex
2. Add Edge
3. Display Connections
4. Exit
Enter your choice: 1
Enter vertex to add: 14
Vertex 14 added.

1. Add Vertex
2. Add Edge
3. Display Connections
4. Exit
Enter your choice: 2
Enter first vertex: 11
Enter second vertex: 14
One or both vertices do not exist.

1. Add Vertex
2. Add Edge
3. Display Connections
4. Exit
```

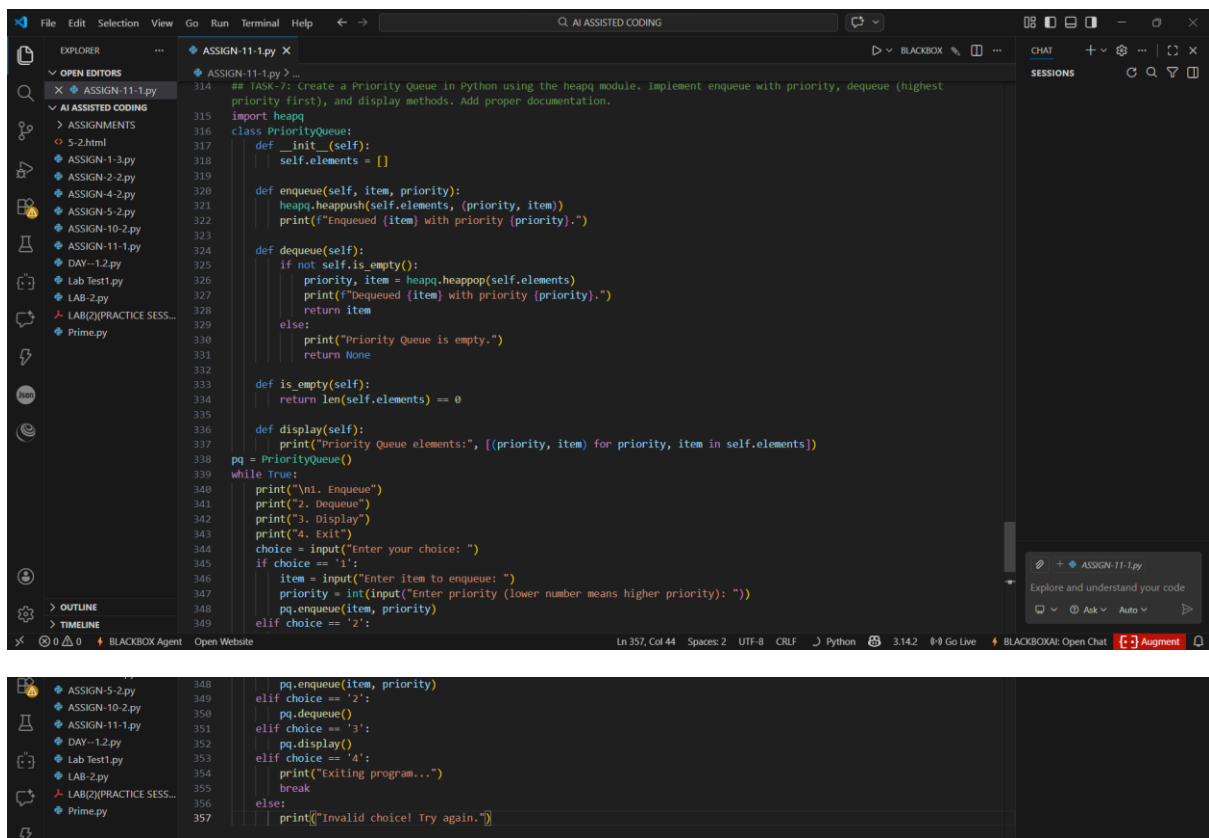
EXPLANATION: A Graph is a non-linear data structure used to represent relationships between entities. It consists of vertices (nodes) and edges (connections). Graphs are commonly used in networks, maps, and routing systems.

Task Description #7 – Priority Queue:

Task: Use AI to implement a priority queue using Python’s heapq module.

PROMPT: Create a Priority Queue in Python using the heapq module. Implement enqueue with priority, dequeue (highest priority first), and display methods. Add proper documentation.

Sample Input Code:



```
314  ## TASK-7: Create a Priority Queue in Python using the heapq module. Implement enqueue with priority, dequeue (highest
315  priority first), and display methods. Add proper documentation.
316  import heapq
317  class PriorityQueue:
318      def __init__(self):
319          self.elements = []
320
321      def enqueue(self, item, priority):
322          heapq.heappush(self.elements, (priority, item))
323          print(f"Enqueued {item} with priority {priority}.")
324
325      def dequeue(self):
326          if not self.is_empty():
327              priority, item = heapq.heappop(self.elements)
328              print(f"Dequeued {item} with priority {priority}.")
329              return item
330          else:
331              print("Priority Queue is empty.")
332              return None
333
334      def is_empty(self):
335          return len(self.elements) == 0
336
337      def display(self):
338          print("Priority Queue elements:", [(priority, item) for priority, item in self.elements])
339
340  pq = PriorityQueue()
341  while True:
342      print("\n1. Enqueue")
343      print("2. Dequeue")
344      print("3. Display")
345      print("4. Exit")
346      choice = input("Enter your choice: ")
347      if choice == '1':
348          item = input("Enter item to enqueue: ")
349          priority = int(input("Enter priority (lower number means higher priority): "))
350          pq.enqueue(item, priority)
351      elif choice == '2':
352          pq.dequeue()
353      elif choice == '3':
354          pq.display()
355      elif choice == '4':
356          print("Exiting program...")
357          break
358      else:
359          print("Invalid choice! Try again.")
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\ASSIGN-11-1.py"
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter item to enqueue: 11
Enter priority (lower number means higher priority): 14
Enqueued 11 with priority 14.

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued 11 with priority 14.

1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exit
```

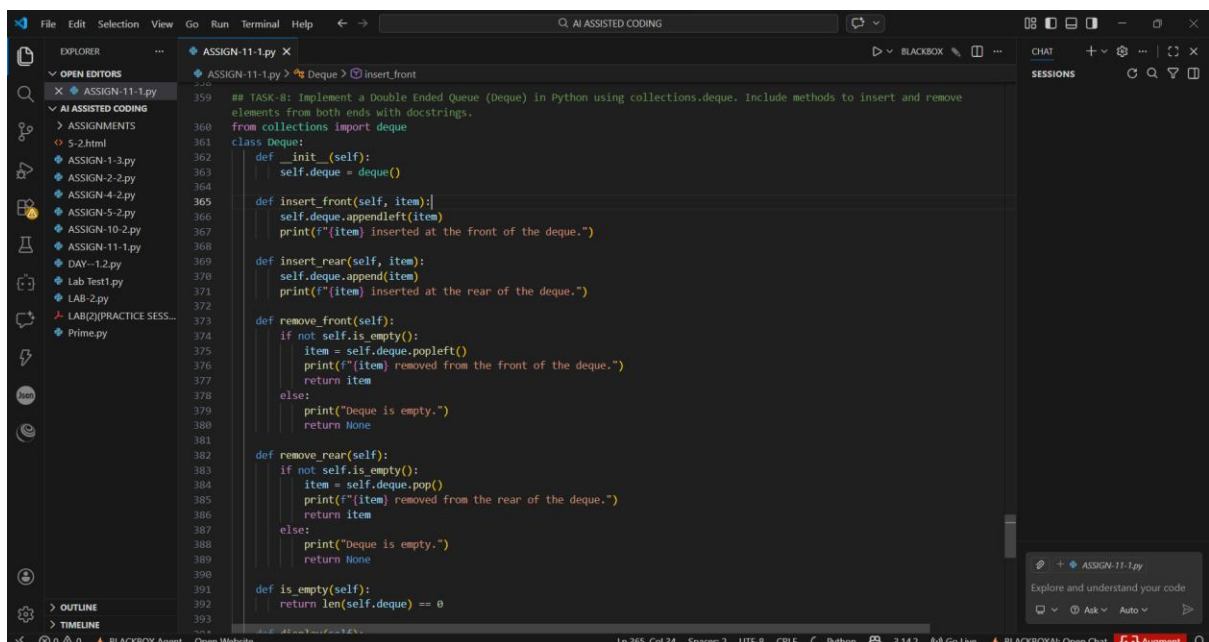
EXPLANATION: A Priority Queue is a special type of queue where elements are removed based on priority rather than order of insertion. Higher priority elements are processed first. It is typically implemented using a heap for efficiency.

Task Description #8 – Deque:

Task: Use AI to implement a double-ended queue using `Collections.deque`.

PROMPT: Implement a Double Ended Queue (Deque) in Python using `collections.deque`. Include methods to insert and remove elements from both ends with docstrings.

Sample Input Code:



```
## TASK-8: Implement a Double Ended Queue (Deque) in Python using collections.deque. Include methods to insert and remove
elements from both ends with docstrings.
from collections import deque
class Deque:
    def __init__(self):
        self.deque = deque()

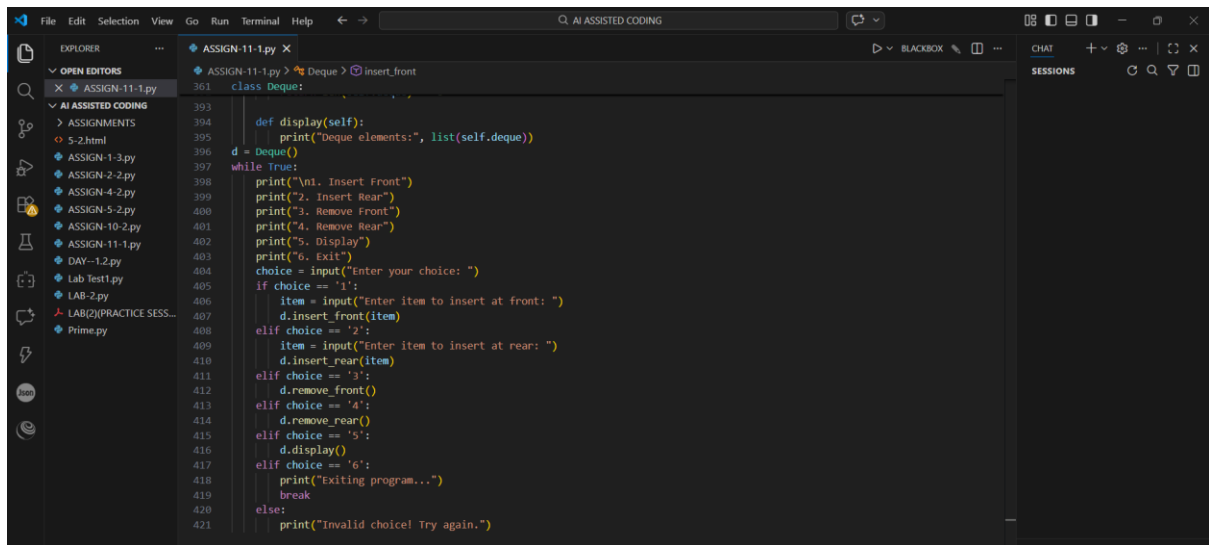
    def insert_front(self, item):
        self.deque.appendleft(item)
        print(f"{item} Inserted at the front of the deque.")

    def insert_rear(self, item):
        self.deque.append(item)
        print(f"{item} Inserted at the rear of the deque.")

    def remove_front(self):
        if not self.is_empty():
            item = self.deque.popleft()
            print(f"{item} removed from the front of the deque.")
            return item
        else:
            print("Deque is empty.")
            return None

    def remove_rear(self):
        if not self.is_empty():
            item = self.deque.pop()
            print(f"{item} removed from the rear of the deque.")
            return item
        else:
            print("Deque is empty.")
            return None

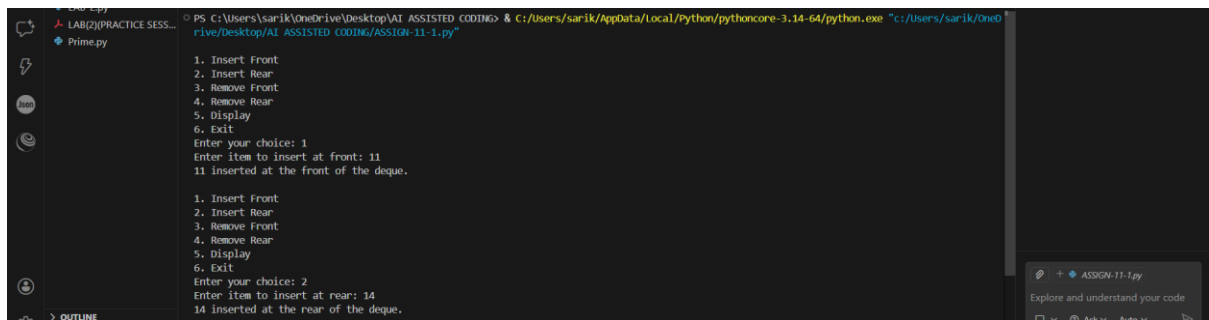
    def is_empty(self):
        return len(self.deque) == 0
```



The screenshot shows a code editor with a file explorer on the left and a chat panel on the right. The main editor displays the code for a Deque class in Python. The code includes methods for inserting and removing elements from both ends, displaying the deque, and exiting the program.

```
361 class Deque:
362     def __init__(self):
363         self.items = []
364     def display(self):
365         print("Deque elements:", list(self.deque))
366     d = Deque()
367     while True:
368         print("\n1. Insert Front")
369         print("\n2. Insert Rear")
370         print("\n3. Remove Front")
371         print("\n4. Remove Rear")
372         print("\n5. Display")
373         print("\n6. Exit")
374         choice = input("Enter your choice: ")
375         if choice == '1':
376             item = input("Enter item to insert at front: ")
377             d.insert_front(item)
378         elif choice == '2':
379             item = input("Enter item to insert at rear: ")
380             d.insert_rear(item)
381         elif choice == '3':
382             d.remove_front()
383         elif choice == '4':
384             d.remove_rear()
385         elif choice == '5':
386             d.display()
387         elif choice == '6':
388             print("Exiting program...")
389             break
390         else:
391             print("Invalid choice! Try again.")
```

OUTPUT:



The screenshot shows a terminal window with the output of the Deque program. The user has entered choices 1, 2, and 5, and the program has responded accordingly, showing the deque's state after each operation.

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe -c "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-11-1.py"
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 1
Enter item to insert at front: 11
11 inserted at the front of the deque.

1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter item to insert at rear: 14
14 inserted at the rear of the deque.
```

EXPLANATION: A Deque (Double Ended Queue) allows insertion and deletion of elements from both the front and rear ends. It combines features of both stacks and queues. It is useful in applications like sliding window algorithms and task scheduling.